

Recipe Book

Android & Firebase Application

Project Completion Report

1. Executive Summary

Recipe Book is a native Android application that lets users register, browse, share, and manage cooking recipes on a Firebase-backed social platform. This report documents the completed engineering work across delivery phases, covering project scaffolding, authentication, the recipe feed, full recipe CRUD with image hosting, and the recipe details experience.

The application was built following Clean Architecture and MVVM, with a strict separation between a framework-free domain layer, a Firebase/Cloudinary-backed data layer, and a Fragment/ViewModel presentation layer. All backend services run on free-tier infrastructure: Firebase Authentication and Firestore for accounts and data, and Cloudinary for image hosting .

2. Project Overview

Recipe Book allows users to securely register and log in, post original recipes with images and instructions, browse a searchable and filterable feed, manage their own contributions, and view rich recipe details including a linked YouTube video.

2.1 Core Features (Original Specification)

✓	Register to the application
✓	Login and optionally save login information ("Remember me")
✓	Discover recipes shared by all users
✓	Create and share recipes, with an optional photo
✓	Update recipes the user created
✓	Delete recipes the user created
✓	Search recipes by name, scoped to the active category
○	View all recipes created by the user on a dedicated Profile screen (Phase 5)
✓	View recipe details: ingredients, steps, and a linked YouTube video
✓	Logout from the current account
✓	Interactive feedback on every user action (Toast/Snackbar confirmations)

3. Architecture

The application follows Clean Architecture with three strictly layered modules, matching the separation of concerns.

Layer	Backend Analogy	Responsibility
Presentation (Fragment + ViewModel)	Controller	Renders UI, reacts to input, holds UI state via <code>StateFlow<UiState<T>></code>
Domain (UseCases, models, Repository interfaces)	Service layer	Framework-free business logic and validation rules

Layer	Backend Analogy	Responsibility
Data (Repository impls, Firebase/Cloudinary data sources, DTOs, mappers)	Repository / DAO	Talks to Firebase & Cloudinary, maps DTOs to domain models

Dependency direction is strictly inward: the data layer implements domain interfaces, the presentation layer depends only on domain interfaces and use cases, and the domain layer has zero Android framework dependencies — verified in practice when a stray `android.util.Patterns` import was caught and replaced with a pure Kotlin regex so that domain-layer unit tests run on the plain JVM without an emulator or Robolectric.

4. Technology Stack

Layer	Choice	Why
Language	Kotlin	Null-safety, coroutines for structured async code
UI Toolkit	XML Views + View Binding	Traditional, well-documented path suited to a first Android project
Architecture	Clean Architecture + MVVM	Mirrors familiar Controller/Service/Repository layering
Async	Coroutines + Flow	Structured concurrency; Flow as the reactive-stream equivalent
DI	Hilt (Dagger)	Compile-time injection, Spring-like ergonomics
Auth & Database	Firebase Authentication + Firestore (Spark/free)	Managed auth, real-time NoSQL document store, zero backend hosting
Image Hosting	Cloudinary (free tier)	Firebase Storage now requires the paid Blaze plan (Feb 2026); Cloudinary keeps the stack at \$0
Image Loading	Glide	Caching, placeholder/error states
Navigation	Jetpack Navigation Component + Safe Args	Type-safe args, single-Activity architecture
Session Storage	DataStore Preferences	Coroutine-friendly "Remember me" persistence
Lists	ListAdapter + DiffUtil	Efficient diffing, no full-list re-renders
Testing	JUnit (+ hand-written fakes)	Fast, emulator-free unit tests for validators and use cases

5. Implementation Summary by Phase

Phase 0 — Project Setup

Established the Gradle project (Kotlin, Hilt, Firebase BoM, Navigation, Glide, DataStore, Cloudinary SDK), AndroidManifest, application/theme scaffolding, and a temporary Splash destination proving the full dependency graph (Hilt → Firebase → Cloudinary) initializes cleanly before any feature work began.

Phase 1 — Authentication

- AuthRepository (domain interface) + Firebase-backed implementation, with Register/Login/Logout/CheckSession use cases
- Login and Register screens with full inline validation and Snackbar error handling
- "Remember me" session persistence via Jetpack DataStore; Splash screen performs a real session check and routes to Login or Home
- Rollback safety: if the Firestore profile write fails immediately after Firebase Auth account creation, the orphaned Auth account is deleted rather than left without a profile
- Never reveal which of email/password was wrong on a failed login — avoids user enumeration

Phase 2 — Recipe Feed

- Real-time Firestore listener wrapped as a Kotlin Flow (callbackFlow), automatically unsubscribed when the Fragment's view is destroyed
- Dynamic category tabs derived from live data, with category names normalized to Title Case to avoid duplicate tabs from casing differences
- Loading / Empty / Error / Success states for the feed, each with dedicated UI
- Lightweight client-side title search layered on top of the server-side category query — deliberately pulled forward from Phase 4 once the category-filtered feed already existed
- ListAdapter + DiffUtil recipe cards with Glide image loading and placeholder/error handling

Phase 3 — Add / Edit / Delete Recipes

- Full Cloudinary image pipeline: local compression → unsigned upload → Firestore write with the resulting secure_url, in that strict order
- A locally pre-generated Firestore document ID doubles as the Cloudinary public_id, so re-uploading during an edit cleanly overwrites the previous image instead of orphaning assets
- One screen (AddEditRecipeFragment) serves both Add and Edit modes via a nullable Safe Args argument
- Owner-only Edit/Delete icons on recipe cards — a client-side UX convenience; the real enforcement is the Firestore Security Rule requiring resource.data.authorId == request.auth.uid
- Delete requires an explicit confirmation dialog before any destructive call
- Picked-image state lives in the ViewModel (not the Fragment), so rotating the device mid-form does not lose the user's photo selection

Phase 4 — Search Polish & Recipe Details

- Search input now debounced at 300ms via a cancelable Handler callback, avoiding a filter pass on every keystroke
- Recipe Details screen with a CollapsingToolbarLayout hero image, category chip, author, and a "Watch on YouTube" button that opens an implicit ACTION_VIEW intent
- Ingredients and steps render in height-capped, internally scrollable text blocks (~7 lines before scrolling), avoiding nested-scroll gesture conflicts with the screen's own NestedScrollView

- Owner-only Edit/Delete menu items in the Details toolbar, with the same confirmation-dialog delete flow as the feed
- The screen refreshes its data on every resume, so editing a recipe and navigating back shows the latest content rather than stale data from the initial one-shot fetch

Phase 5 — User Profile & Refinement

- Profile Picture Updates: Users can now update their profile photo directly from the Profile screen.
- Unique Asset Strategy: To resolve "overwrite" restrictions in Cloudinary unsigned uploads, the app generates unique asset IDs, ensuring every update is successful and cache-free.
- Registration Spinner: Replaced manual country entry with a validated dropdown menu for improved data consistency.

• 6. Firebase & Cloudinary Integration

6.1 Firestore Schema

Collection	Key Fields	Notes
users/{userId}	name, email, photoUrl, country, createdAt	Mirrors Firebase Auth's uid as the document ID
recipes/{recipeId}	title, titleLower, ingredients[], steps[], category, imageUrl, videoUrl, authorId, authorName, createdAt	authorName is intentionally denormalized — Firestore has no JOINS, so this avoids an N+1 read per list item

6.2 Security Rules

Firestore Security Rules — not client-side checks — are the actual authorization boundary. Reads require an authenticated session; recipe creation requires the authenticated uid to match the document's authorId; updates and deletes require the same match against the existing document's stored authorId. Account documents can be created and updated only by their owner and can never be deleted via the client in this version.

6.3 Why Cloudinary Instead of Firebase Storage

Firebase Cloud Storage requires the paid Blaze plan even to provision a bucket. Rather than link a billing account for a portfolio-scale project, image hosting was moved to Cloudinary's free tier, using unsigned upload presets scoped to a dedicated folder. Firestore's imageUrl field stores the resulting URL exactly as it would have stored a Firebase Storage download URL — no other part of the architecture changed.

8. Security Considerations

- Passwords are never handled or stored by the application — Firebase Authentication owns that entirely

- Firestore Security Rules enforce ownership server-side; the app's owner-only UI affordances are a convenience layer only
- Cloudinary's unsigned upload preset is a deliberate, documented trade-off for a project at this scale — a production app handling sensitive user content would move to signed uploads generated by a small server-side function
- Client-side validation never substitutes for server-side enforcement anywhere in the codebase

11. Conclusion

Recipe Book demonstrates a complete, production-patterned Android application built from first principles: Clean Architecture with a genuinely framework-free domain layer, MVVM state management via Kotlin Flow, a fully server-enforced security model, and a pragmatic real-world adaptation (the Cloudinary migration) made in direct response to a mid-project platform policy change. Phases 0 through 5 deliver a working, testable application covering registration, authentication, recipe discovery, full recipe CRUD with image hosting, and a polished recipe details experience. The remaining phases are scoped and ready to pick up as a continuation of the same disciplined, phase-by-phase delivery process used throughout.